

Report Lab9 Transectional

1. การสร้าง Account

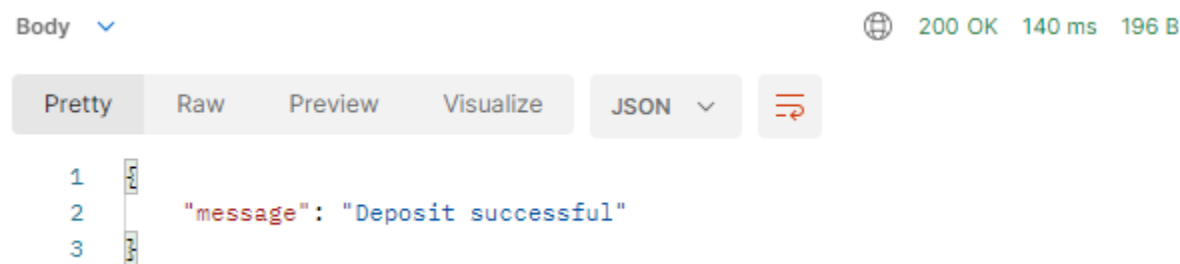
POST <http://localhost:8081/accounts>



ระบบสามารถสร้าง Account ได้สำเร็จ และกำหนดค่า id ให้กับบัญชีโดยอัตโนมัติ จากนั้นสามารถนำ id ที่ได้ไปใช้ในการทำรายการฝากเงินต่อไป

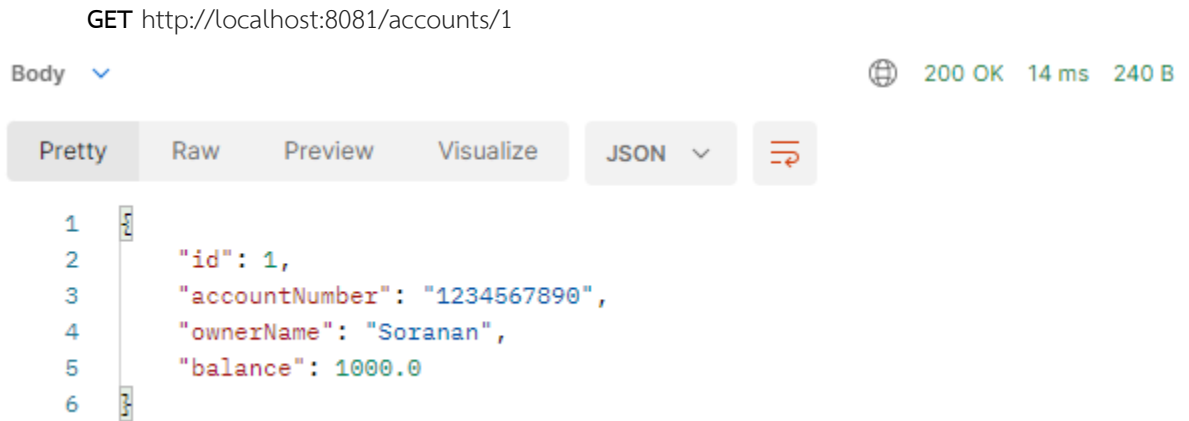
2. การฝากเงิน

POST <http://localhost:8080/accounts/1/deposit>



ระบบสามารถฝากเงินได้สำเร็จ โดยยอดเงินใน Account เพิ่มขึ้นจาก 0 บาท เป็น 1,000 บาท และมีข้อมูลประวัติการฝากเงินถูกบันทึกลงในตาราง deposit_transaction

3. ตรวจสอบยอดเงิน



พบว่ายอดเงินในบัญชีมีการเปลี่ยนแปลงตามจำนวนเงินที่ฝาก เช่น จากเดิม 0 บาท เพิ่มขึ้น 1,000 บาท แสดงให้เห็นว่าการเพิ่มยอดเงินใน Account สามารถทำงานได้อย่างถูกต้อง

4. ตรวจสอบประวัติการฝากเงิน

Data Output			
Messages			
Notifications			
	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

ผลการทดลองพบว่าข้อมูลรายการฝากเงินถูกบันทึก โดยประกอบด้วยข้อมูลสำคัญ ได้แก่ id, amount และ account_id

5. ทดลอง Transaction Rollback

POST http://localhost:8081/accounts/1/deposit

Body 500 Internal Server Error 417 ms 273 B

Pretty Raw Preview Visualize JSON ⌵

```
1 {
2   "timestamp": "2026-09-11T07:17:33.616+00:00",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/1/deposit"
6 }
```

@Transactional ทำให้การเพิ่มยอดเงินและการบันทึกประวัติการฝากเงินถูกควบคุมให้อยู่ภายใน Transaction เดียวกัน หากขั้นตอนใดขั้นตอนหนึ่งเกิดข้อผิดพลาด ระบบจะยกเลิกการเปลี่ยนแปลงทั้งหมด

6. ตรวจสอบว่าข้อมูลถูกย้อนกลับไหม

GET <http://localhost:8081/accounts/1>

Body 200 OK 31 ms 240 B

Pretty Raw Preview Visualize JSON ⌵

```
1 {
2   "id": 1,
3   "accountNumber": "1234567890",
4   "ownerName": "Soranan",
5   "balance": 1000.0
6 }
```

Data Output Messages Notifications

	Id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1

ผลที่ถูกต้องยังเป็น 1000 เพราะ Transaction ถูก Rollback และ ตาราง deposit_transaction ยังมี 1 แถวเท่าเดิม Response จาก Postman ที่แสดงว่า balance ไม่เพิ่มและไม่มีเพิ่มรายการใหม่

7. ทดลองโดยเอา @Transactional ออก



ให้ลบ @Transactional ออก โดยยังคงบรรทัด `throw new RuntimeException("Test Rollback");` ไว้เหมือนเดิม แล้วลองฝากเงินอีกครั้ง ได้ 500 Internal Server Error เมื่อไม่มี Transaction ครอบคลุม การเปลี่ยนแปลงที่เกิดขึ้นก่อน Exception อาจไม่ถูกย้อนกลับ

8. ผลการทดลองที่แสดงความแตกต่าง

8.1 เมื่อมี @Transactional

คำสั่งเพิ่มยอดเงินและบันทึกประวัติการฝากเงินจะถูกจัดให้อยู่ใน Transaction เดียวกัน หากเกิดข้อผิดพลาดระหว่างการทำงาน ระบบจะทำการ Rollback การเปลี่ยนแปลงทั้งหมด ทำให้ข้อมูลกลับไปเป็นสถานะก่อนเริ่มฝากเงิน

Data Output				Messages	Notifications
	id [PK] bigint	amount double precision	account_id bigint		
1	1	1000	1		

8.2 เมื่อไม่มี @Transactional

การเปลี่ยนแปลงแต่ละส่วนไม่ได้ถูกควบคุมให้อยู่ใน Transaction เดียวกัน เมื่อเกิดข้อผิดพลาด ข้อมูลบางส่วนอาจถูกบันทึกไปแล้วและไม่ถูกยกเลิก ส่งผลให้ ยอดเงินกับประวัติการฝากเงินอาจไม่สอดคล้องกัน

Data Output Messages Notifications			
       			
	id [PK] bigint	amount double precision	account_id bigint
1	1	1000	1
2	3	1000	1

9. คำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

@Transactional มีหน้าที่กำหนดขอบเขตของ Transaction ให้งานในเมธอด โดยคำสั่งที่อยู่ภายใน Transaction จะถูกจัดการเป็นกลุ่มเดียวกัน หากทำงานสำเร็จทั้งหมดจะทำการ COMMIT แต่หากเกิดข้อผิดพลาดหรือ Exception ระบบจะทำการ ROLLBACK การเปลี่ยนแปลงทั้งหมดภายใน Transaction

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เนื่องจากการฝากเงินประกอบด้วยหลายขั้นตอน เช่น การเพิ่มยอดเงินใน Account และการบันทึกประวัติลงใน DepositTransaction ซึ่งทั้งสองขั้นตอนต้องทำงานสอดคล้องกัน หากขั้นตอนใดเกิดข้อผิดพลาด ควรยกเลิกการเปลี่ยนแปลงทั้งหมด เพื่อป้องกันข้อมูลไม่ถูกต้อง เช่น ยอดเงินเพิ่มขึ้นแต่ไม่มีประวัติการฝากเงิน ดังนั้นจึงควรใช้ Transaction เพื่อให้การทำงานเป็น ทั้งสำเร็จหรือทั้งหมดถูกยกเลิก

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือ การยืนยันการเปลี่ยนแปลงข้อมูลใน Transaction และบันทึกข้อมูลลง Database อย่างถาวร

ROLLBACK คือ การยกเลิกการเปลี่ยนแปลงที่เกิดขึ้นภายใน Transaction และทำให้ข้อมูลกลับไปอยู่ในสถานะก่อนเริ่ม Transaction

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

Account และ **DepositTransaction** มีความสัมพันธ์แบบ One-to-Many (1:N) โดย Account หนึ่งบัญชีสามารถมีรายการฝากเงิน (**DepositTransaction**) ได้หลายรายการ ในขณะที่ DepositTransaction แต่ละรายการจะเป็นของ Account เพียงหนึ่งบัญชี

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

เมื่อเกิด Error หรือ Exception ภายใน Transaction ระบบจะทำการ **ROLLBACK** การเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นภายใน Transaction นั้น เช่น หากเพิ่มยอดเงินแล้วเกิด Error ก่อนจบกระบวนการ ยอดเงินที่เพิ่มและข้อมูลประวัติการฝากเงินจะถูกยกเลิก ทำให้ Database กลับไปเป็นสถานะก่อนเริ่มทำรายการ

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อทดลองลบ **@Transactional** ออก หากเกิด Error ระหว่างการทำงาน ระบบจะไม่มี Transaction เดียวที่ครอบคลุมกระบวนการฝากเงินทั้งหมด ทำให้การเปลี่ยนแปลงบางส่วน อาจถูกบันทึกลง Database ไปแล้ว แม้ว่าขั้นตอนถัดไปจะเกิดข้อผิดพลาดก็ตาม